Codes:

mainActivity.kt

```
package com.example.taskmanager

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.Checklist
import androidx.compose.material.icons.filled.Settings
import androidx.compose.material.icons.filled.TaskAlt
import androidx.compose.material.icons.outlined.Checklist
import androidx.compose.material.icons.outlined.Logout
import androidx.compose.material.icons.outlined.Person
import androidx.compose.material.icons.outlined.TaskAlt
import androidx.compose.material3.DropdownMenu
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FloatingActionButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.NavigationBar
import androidx.compose.material3.NavigationBarItem
import androidx.compose.material3.NavigationBarItemDefaults
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
```

```
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavHostController
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.currentBackStackEntryAsState
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.example.taskmanager.navigation.Screen
import com.example.taskmanager.ui.screens.AddEditTaskScreen
import com.example.taskmanager.ui.screens.ManageScreen
import com.example.taskmanager.ui.screens.ProfileScreen
import com.example.taskmanager.ui.screens.SignInScreen
import com.example.taskmanager.ui.screens.SignUpScreen
import com.example.taskmanager.ui.screens.TaskListScreen
import com.example.taskmanager.ui.theme.CyanAccent
import com.example.taskmanager.ui.theme.DarkBackground
import com.example.taskmanager.ui.theme.DarkCard
import com.example.taskmanager.ui.theme.DarkSurface
import com.example.taskmanager.ui.theme.NeonPink
import com.example.taskmanager.ui.theme.PurpleAccent
import com.example.taskmanager.ui.theme.PurpleDark
import com.example.taskmanager.ui.theme.TaskManagerTheme
import com.example.taskmanager.ui.theme.TextDimGrey
import com.example.taskmanager.ui.theme.TextGrey
import com.example.taskmanager.viewmodel.AuthViewModel
import com.example.taskmanager.viewmodel.TaskViewModel
import com.example.taskmanager.viewmodel.TaskViewModelFactory

class MainActivity : ComponentActivity() {
   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)
       enableEdgeToEdge()
       setContent {
           TaskManagerTheme {
               val app = application as TaskManagerApp
               val factory = TaskViewModelFactory(app.repository)
               val taskViewModel: TaskViewModel = viewModel(factory = factory)
               val authViewModel: AuthViewModel = viewModel(factory = factory)
               TaskManagerMainScreen(
                   taskViewModel = taskViewModel,
                   authViewModel = authViewModel
               )
           }
       }
   }
}

data class BottomNavItem(
```

```
    val label: String,
    val route: String,
    val selectedIcon: ImageVector,
    val unselectedIcon: ImageVector
)

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TaskManagerMainScreen(
    taskViewModel: TaskViewModel,
    authViewModel: AuthViewModel
) {
    val navController = rememberNavController()
    val navBackStackEntry by navController.currentBackStackEntryAsState()
    val currentRoute = navBackStackEntry?.destination?.route

    val currentUser by authViewModel.currentUser.collectAsState()

    // Sync userId to TaskViewModel whenever user changes
    LaunchedEffect(currentUser) {
        taskViewModel.setUserId(currentUser?.userId ?: 0)
    }

    var showSettingsMenu by remember { mutableStateOf(false) }

    val bottomNavItems = listOf(
        BottomNavItem(
            label = "Tasks",
            route = Screen.TaskList.route,
            selectedIcon = Icons.Filled.TaskAlt,
            unselectedIcon = Icons.Outlined.TaskAlt
        ),
        BottomNavItem(
            label = "Manage",
            route = Screen.Manage.route,
            selectedIcon = Icons.Filled.Checklist,
            unselectedIcon = Icons.Outlined.Checklist
        )
    )

    // Auth screens don't show bottom nav or FAB
    val authScreens = listOf(Screen.SignIn.route, Screen.SignUp.route)
    val isAuthScreen = currentRoute in authScreens
    val isMainScreen = currentRoute in listOf(Screen.TaskList.route,
Screen.Manage.route)

    Scaffold(
        modifier = Modifier.fillMaxSize(),
        containerColor = DarkBackground,
```

```
        topBar = {
            // TopBar always visible across ALL screens
            TopAppBar(
                title = {
                    Text(
                        text = "Task Manager",
                        style = MaterialTheme.typography.titleLarge,
                        fontWeight = FontWeight.Bold,
                        color = Color.White
                    )
                },
                actions = {
                    if (!isAuthScreen) {
                        Box {
                            IconButton(onClick = { showSettingsMenu = true }) {
                                Icon(
                                    imageVector = Icons.Filled.Settings,
                                    contentDescription = "Settings",
                                    tint = PurpleAccent
                                )
                            }
                            DropdownMenu(
                                expanded = showSettingsMenu,
                                onDismissRequest = { showSettingsMenu = false },
                                containerColor = DarkCard
                            ) {
                                DropdownMenuItem(
                                    text = {
                                        Text("Profile", color = Color.White)
                                    },
                                    onClick = {
                                        showSettingsMenu = false

navController.navigate(Screen.Profile.route) {
                                            launchSingleTop = true
                                        }
                                    },
                                    leadingIcon = {
                                        Icon(
                                            Icons.Outlined.Person,
                                            contentDescription = null,
                                            tint = CyanAccent
                                        )
                                    }
                                )
                                DropdownMenuItem(
                                    text = {
                                        Text("Logout", color = NeonPink)
                                    },
```

```
                                    onClick = {
                                         showSettingsMenu = false
                                        authViewModel.logout()

navController.navigate(Screen.SignIn.route) {
                                             popUpTo(0) { inclusive = true }
                                         }
                                     },
                                    leadingIcon = {
                                         Icon(
                                            Icons.Outlined.Logout,
                                            contentDescription = null,
                                            tint = NeonPink
                                         )
                                    }
                                )
                            }
                        }
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = DarkBackground,
                    titleContentColor = Color.White
                )
            )
        },
        bottomBar = {
            if (!isAuthScreen && currentRoute != Screen.Profile.route) {
                NavigationBar(
                    containerColor = DarkSurface,
                    tonalElevation = 8.dp
                ) {
                    bottomNavItems.forEach { item ->
                        val isSelected = currentRoute == item.route
                        NavigationBarItem(
                            selected = isSelected,
                            onClick = {
                                if (currentRoute != item.route) {
                                    navController.navigate(item.route) {
                                         popUpTo(Screen.TaskList.route) {
                                             saveState = true
                                        }
                                         launchSingleTop = true
                                        restoreState = true
                                    }
                                }
                            },
                            icon = {
                                Icon(
```

```
                                    imageVector = if (isSelected)
item.selectedIcon else item.unselectedIcon,
                                    contentDescription = item.label
                                )
                            },
                            label = {
                                 Text(
                                    text = item.label,
                                    style = MaterialTheme.typography.labelSmall,
                                    fontWeight = if (isSelected) FontWeight.Bold
else FontWeight.Normal
                                )
                            },
                            colors = NavigationBarItemDefaults.colors(
                                selectedIconColor = PurpleAccent,
                                selectedTextColor = PurpleAccent,
                                unselectedIconColor = TextDimGrey,
                                unselectedTextColor = TextDimGrey,
                                indicatorColor = PurpleDark.copy(alpha = 0.3f)
                            )
                        )
                    }
                }
            }
        },
        floatingActionButton = {
            if (isMainScreen) {
                FloatingActionButton(
                    onClick = {
                         navController.navigate(Screen.AddTask.route)
                    },
                    containerColor = PurpleDark,
                    contentColor = Color.White,
                    shape = RoundedCornerShape(16.dp),
                    modifier = Modifier.shadow(
                        elevation = 12.dp,
                        shape = RoundedCornerShape(16.dp),
                        ambientColor = PurpleAccent.copy(alpha = 0.4f),
                        spotColor = PurpleAccent.copy(alpha = 0.4f)
                    )
                ) {
                     Icon(
                        imageVector = Icons.Filled.Add,
                        contentDescription = "Add Task"
                    )
                }
            }
        }
    ) { innerPadding ->
```

```
        Box(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .background(DarkBackground)
        ) {
            AppNavHost(
                navController = navController,
                taskViewModel = taskViewModel,
                authViewModel = authViewModel
            )
        }
    }
}

@Composable
fun AppNavHost(
    navController: NavHostController,
    taskViewModel: TaskViewModel,
    authViewModel: AuthViewModel
) {
    NavHost(
        navController = navController,
        startDestination = Screen.SignIn.route
    ) {
        // Auth screens
        composable(Screen.SignIn.route) {
            SignInScreen(
                authViewModel = authViewModel,
                onSignInSuccess = {
                    navController.navigate(Screen.TaskList.route) {
                        popUpTo(Screen.SignIn.route) { inclusive = true }
                    }
                },
                onNavigateToSignUp = {
                    navController.navigate(Screen.SignUp.route)
                }
            )
        }

        composable(Screen.SignUp.route) {
            SignUpScreen(
                authViewModel = authViewModel,
                onSignUpSuccess = {
                    navController.navigate(Screen.TaskList.route) {
                        popUpTo(0) { inclusive = true }
                    }
                },
                onNavigateToSignIn = {
```

```
                navController.popBackStack()
            }
        )
    }

    // Main screens
    composable(Screen.TaskList.route) {
        TaskListScreen(
            viewModel = taskViewModel,
            onEditTask = { taskId ->
                navController.navigate(Screen.EditTask.createRoute(taskId))
            }
        )
    }

    composable(Screen.AddTask.route) {
        AddEditTaskScreen(
            viewModel = taskViewModel,
            taskId = null,
            onTaskSaved = {
                navController.popBackStack()
            }
        )
    }

    composable(
        route = Screen.EditTask.route,
        arguments = listOf(navArgument("taskId") { type = NavType.IntType })
    ) { backStackEntry ->
        val taskId = backStackEntry.arguments?.getInt("taskId")
        AddEditTaskScreen(
            viewModel = taskViewModel,
            taskId = taskId,
            onTaskSaved = {
                navController.popBackStack()
            }
        )
    }

    composable(Screen.Manage.route) {
        ManageScreen(viewModel = taskViewModel)
    }

    composable(Screen.Profile.route) {
        ProfileScreen(
            authViewModel = authViewModel,
            taskViewModel = taskViewModel
        )
    }
```

```
    }
}
```

Task.kt (Entity)

```
package com.example.taskmanager.data

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "tasks")
data class Task(
    @PrimaryKey(autoGenerate = true)
    val taskId: Int = 0,
    val userId: Int = 0,
    val title: String,
    val description: String,
    val dueDate: String,
    val isCompleted: Boolean = false
)
```

TaskDao.kt

```
package com.example.taskmanager.data

import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
import kotlinx.coroutines.flow.Flow

@Dao
interface TaskDao {

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertTask(task: Task)

    @Update
    suspend fun updateTask(task: Task)

    @Delete
    suspend fun deleteTask(task: Task)

    @Query("SELECT * FROM tasks WHERE userId = :userId ORDER BY taskId DESC")
    fun getAllTasksForUser(userId: Int): Flow<List<Task>>
```

```
    @Query("SELECT * FROM tasks WHERE taskId = :taskId")
    fun getTaskById(taskId: Int): Flow<Task?>

    @Query("SELECT COUNT(*) FROM tasks WHERE userId = :userId")
    fun getTaskCountForUser(userId: Int): Flow<Int>
}
```

TAskdatabase.kt:

```
package com.example.taskmanager.data

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [Task::class, User::class], version = 2, exportSchema =
false)
abstract class TaskDatabase : RoomDatabase() {

    abstract fun taskDao(): TaskDao
    abstract fun userDao(): UserDao

    companion object {
        @Volatile
        private var INSTANCE: TaskDatabase? = null

        fun getDatabase(context: Context): TaskDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    TaskDatabase::class.java,
                    "task_database"
                )
                    .fallbackToDestructiveMigration()
                    .build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

TaskRepository.kt

```
package com.example.taskmanager.data

import kotlinx.coroutines.flow.Flow
```

```
class TaskRepository(private val taskDao: TaskDao, private val userDao:
UserDao) {

   // ── Task operations ──
   fun allTasksForUser(userId: Int): Flow<List<Task>> =
taskDao.getAllTasksForUser(userId)

   fun getTaskById(taskId: Int): Flow<Task?> = taskDao.getTaskById(taskId)

   fun getTaskCountForUser(userId: Int): Flow<Int> =
taskDao.getTaskCountForUser(userId)

   suspend fun insert(task: Task) {
       taskDao.insertTask(task)
   }

   suspend fun update(task: Task) {
       taskDao.updateTask(task)
   }

   suspend fun delete(task: Task) {
       taskDao.deleteTask(task)
   }

   // ── User / Auth operations ──
   suspend fun registerUser(user: User): Result<User> {
       return try {
           val existing = userDao.getUserByEmail(user.email)
           if (existing != null) {
               Result.failure(Exception("Email already registered"))
           } else {
               val id = userDao.insertUser(user)
               Result.success(user.copy(userId = id.toInt()))
           }
       } catch (e: Exception) {
           Result.failure(e)
       }
   }

   suspend fun loginUser(email: String, password: String): Result<User> {
       return try {
           val user = userDao.loginUser(email, password)
           if (user != null) {
               Result.success(user)
           } else {
               Result.failure(Exception("Invalid email or password"))
           }
       } catch (e: Exception) {
           Result.failure(e)
```

```
        }
    }

    fun getUserById(userId: Int): Flow<User?> = userDao.getUserById(userId)
}
```

TaskViewModel.kt

```
package com.example.taskmanager.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.taskmanager.data.Task
import com.example.taskmanager.data.TaskRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.flatMapLatest
import kotlinx.coroutines.flow.flowOf
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.launch

class TaskViewModel(private val repository: TaskRepository) : ViewModel() {

    private val _currentUserId = MutableStateFlow(0)

    fun setUserId(userId: Int) {
        _currentUserId.value = userId
    }

    val allTasks: StateFlow<List<Task>> = _currentUserId
        .flatMapLatest { userId ->
            if (userId > 0) repository.allTasksForUser(userId)
            else flowOf(emptyList())
        }
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5000),
            initialValue = emptyList()
        )

    val taskCount: StateFlow<Int> = _currentUserId
        .flatMapLatest { userId ->
            if (userId > 0) repository.getTaskCountForUser(userId)
            else flowOf(0)
        }
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5000),
            initialValue = 0
```

```
        )

    fun getTaskById(taskId: Int) = repository.getTaskById(taskId)
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5000),
            initialValue = null
        )

    fun addTask(title: String, description: String, dueDate: String) {
        viewModelScope.launch {
            repository.insert(
                Task(
                    userId = _currentUserId.value,
                    title = title,
                    description = description,
                    dueDate = dueDate
                )
            )
        }
    }

    fun updateTask(task: Task) {
        viewModelScope.launch {
            repository.update(task)
        }
    }

    fun deleteTask(task: Task) {
        viewModelScope.launch {
            repository.delete(task)
        }
    }
}
```

TaskList UI (LazyColumn)

```
package com.example.taskmanager.ui.screens

import androidx.compose.animation.animateColorAsState
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
```

```
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.CalendarToday
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.Edit
import androidx.compose.material.icons.outlined.TaskAlt
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import com.example.taskmanager.data.Task
import com.example.taskmanager.ui.theme.CyanAccent
import com.example.taskmanager.ui.theme.DarkCard
import com.example.taskmanager.ui.theme.NeonPink
import com.example.taskmanager.ui.theme.PurpleAccent
import com.example.taskmanager.ui.theme.PurpleDark
import com.example.taskmanager.ui.theme.TextDimGrey
import com.example.taskmanager.ui.theme.TextGrey
import com.example.taskmanager.viewmodel.TaskViewModel

@Composable
fun TaskListScreen(
   viewModel: TaskViewModel,
   onEditTask: (Int) -> Unit
) {
   val tasks by viewModel.allTasks.collectAsState()

   if (tasks.isEmpty()) {
       // Empty state
       Box(
```

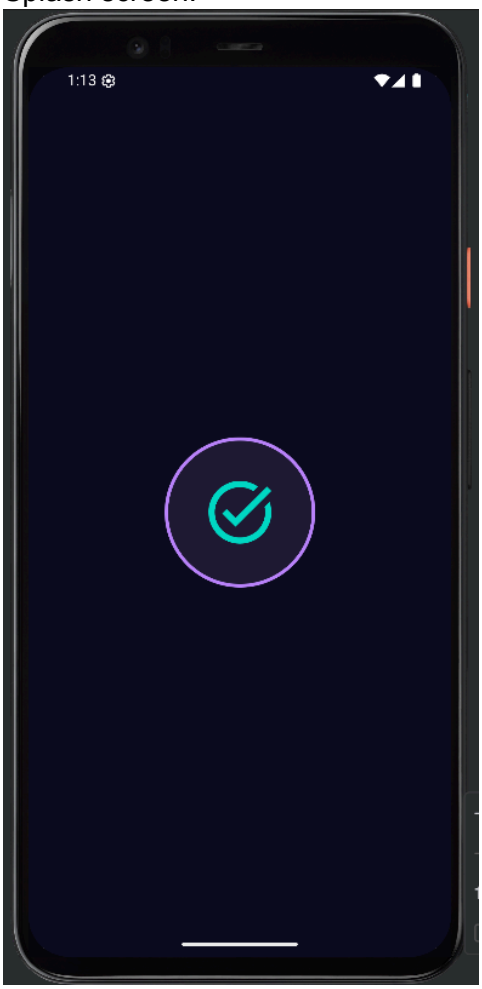
```
            modifier = Modifier.fillMaxSize(),
            contentAlignment = Alignment.Center
        ) {
            Column(horizontalAlignment = Alignment.CenterHorizontally) {
                Icon(
                    imageVector = Icons.Outlined.TaskAlt,
                    contentDescription = null,
                    modifier = Modifier.size(80.dp),
                    tint = PurpleDark.copy(alpha = 0.5f)
                )
                Spacer(modifier = Modifier.height(16.dp))
                Text(
                    text = "No tasks yet",
                    style = MaterialTheme.typography.headlineMedium,
                    color = TextGrey
                )
                Spacer(modifier = Modifier.height(8.dp))
                Text(
                    text = "Tap + to create your first task",
                    style = MaterialTheme.typography.bodyMedium,
                    color = TextDimGrey
                )
            }
        }
    } else {
        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .padding(horizontal = 16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            item { Spacer(modifier = Modifier.height(8.dp)) }
            items(tasks, key = { it.taskId }) { task ->
                TaskCard(
                    task = task,
                    onEdit = { onEditTask(task.taskId) },
                    onDelete = { viewModel.deleteTask(task) }
                )
            }
            item { Spacer(modifier = Modifier.height(80.dp)) }
        }
    }
}

@Composable
fun TaskCard(
    task: Task,
    onEdit: () -> Unit,
    onDelete: () -> Unit
```

```
) {
    val borderGradient = Brush.linearGradient(
        colors = listOf(
            PurpleAccent.copy(alpha = 0.6f),
            CyanAccent.copy(alpha = 0.3f),
            PurpleDark.copy(alpha = 0.2f)
        )
    )

    Card(
        modifier = Modifier
            .fillMaxWidth()
            .clip(RoundedCornerShape(16.dp))
            .border(
                width = 1.dp,
                brush = borderGradient,
                shape = RoundedCornerShape(16.dp)
            ),
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = DarkCard
        ),
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp)
        ) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(
                    text = task.title,
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.Bold,
                    color = Color.White,
                    maxLines = 1,
                    overflow = TextOverflow.Ellipsis,
                    modifier = Modifier.weight(1f)
                )
                Row {
                     IconButton(onClick = onEdit, modifier =
Modifier.size(36.dp)) {
                        Icon(
                            imageVector = Icons.Filled.Edit,
                            contentDescription = "Edit",
```

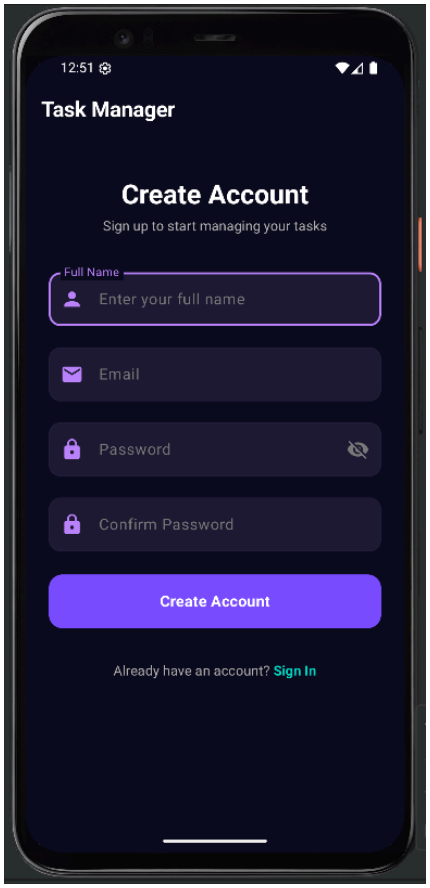
```
                            tint = CyanAccent,
                            modifier = Modifier.size(20.dp)
                        )
                    }
                    IconButton(onClick = onDelete, modifier =
Modifier.size(36.dp)) {
                        Icon(
                            imageVector = Icons.Filled.Delete,
                            contentDescription = "Delete",
                            tint = NeonPink,
                            modifier = Modifier.size(20.dp)
                        )
                    }
                }
            }

            Spacer(modifier = Modifier.height(6.dp))

            Text(
                text = task.description,
                style = MaterialTheme.typography.bodyMedium,
                color = TextGrey,
                maxLines = 2,
                overflow = TextOverflow.Ellipsis
            )

            Spacer(modifier = Modifier.height(10.dp))

            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(
                    imageVector = Icons.Filled.CalendarToday,
                    contentDescription = null,
                    tint = PurpleAccent,
                    modifier = Modifier.size(14.dp)
                )
                Spacer(modifier = Modifier.width(6.dp))
                Text(
                    text = task.dueDate,
                    style = MaterialTheme.typography.bodySmall,
                    color = PurpleAccent
                )
            }
        }
    }
}
```

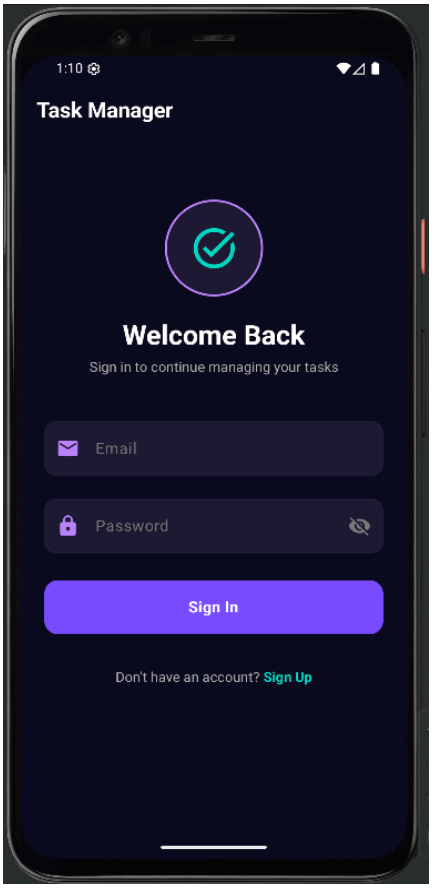
UI screens:

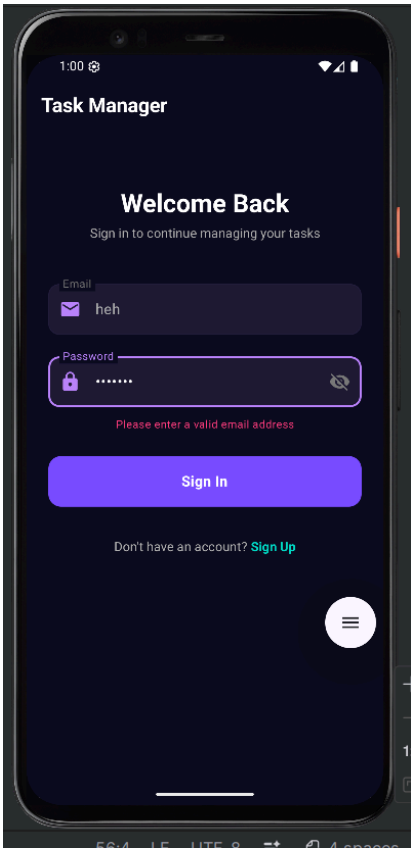
Splash screen:



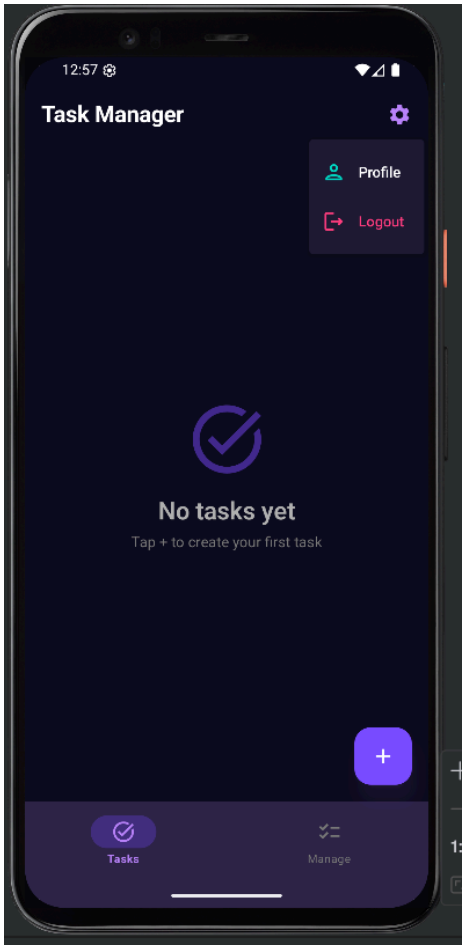
Registration Page and login page:(with validation)


12:51
Task Manager
Create Account
Sign up to start managing your tasks
Full Name
Enter your full name
Email
Password
Confirm Password
Create Account
Already have an account? Sign In

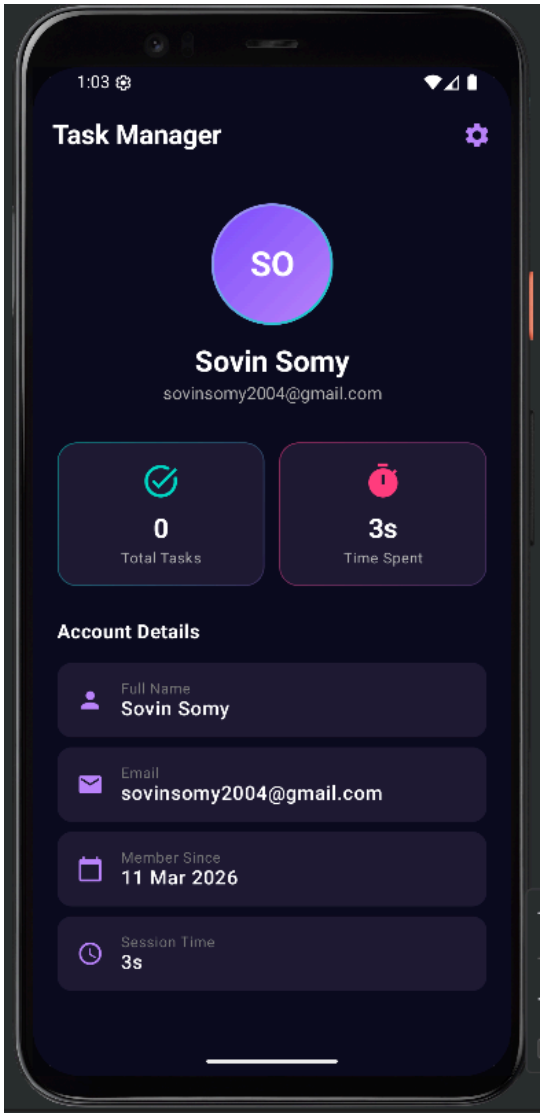

1:10
Task Manager
Welcome Back
Sign in to continue managing your tasks
Email
Password
Sign In
Don't have an account? Sign Up


1:00
Task Manager
Welcome Back
Sign in to continue managing your tasks
Email
heh
Password
Please enter a valid email address
Sign In
Don't have an account? Sign Up

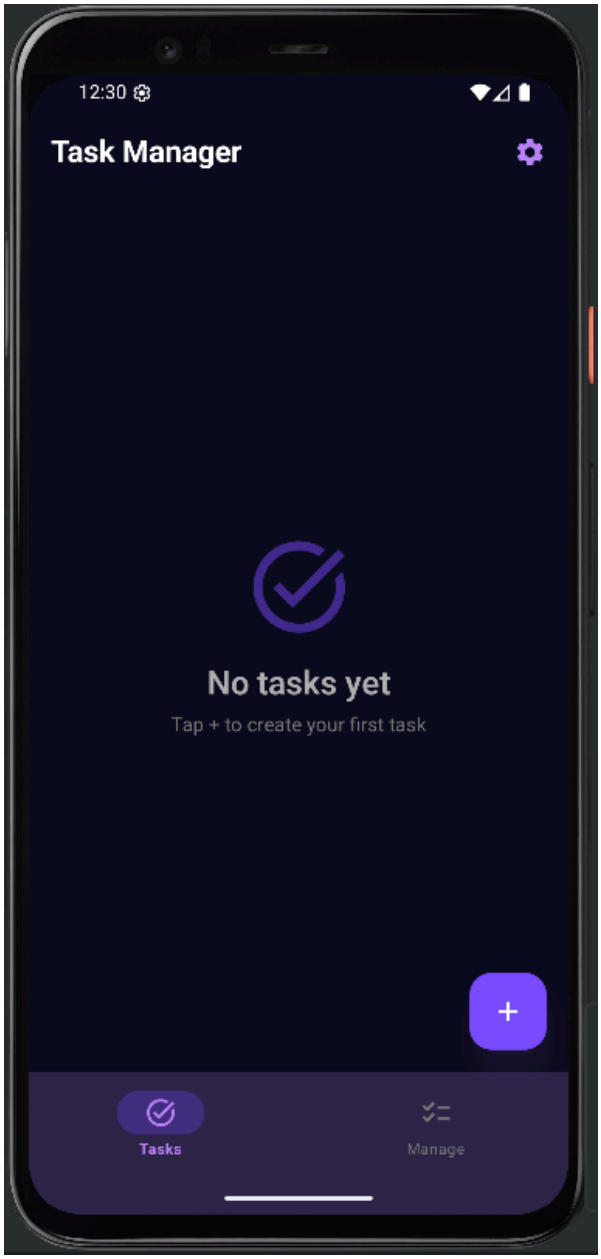
Clickable settings icon:



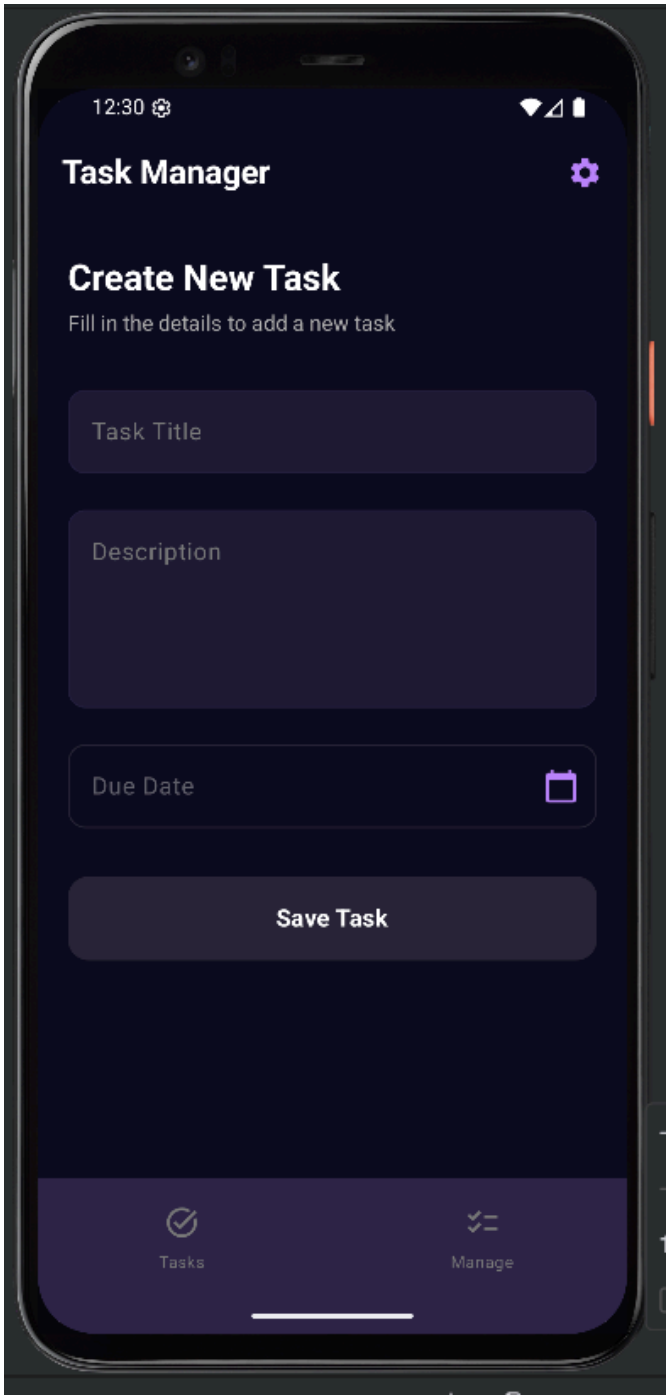
Profile page:



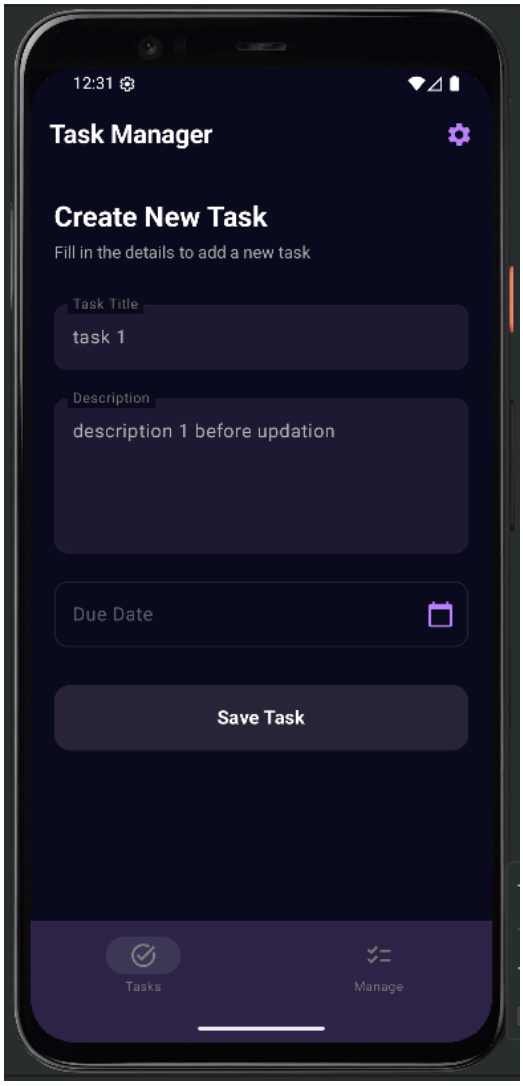
Home screen:

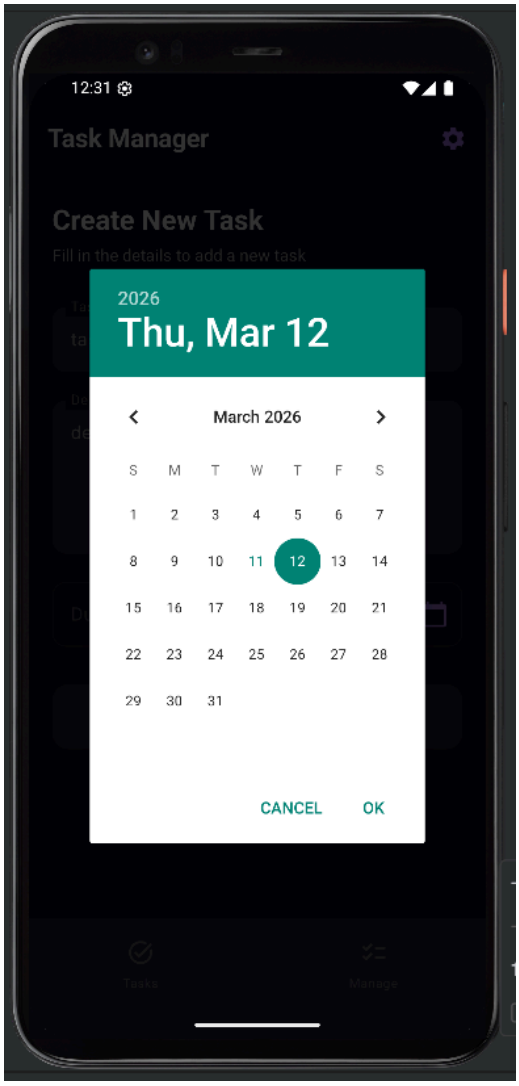

12:30
Task Manager
No tasks yet
Tap + to create your first task
Tasks
Manage

Task Creation:

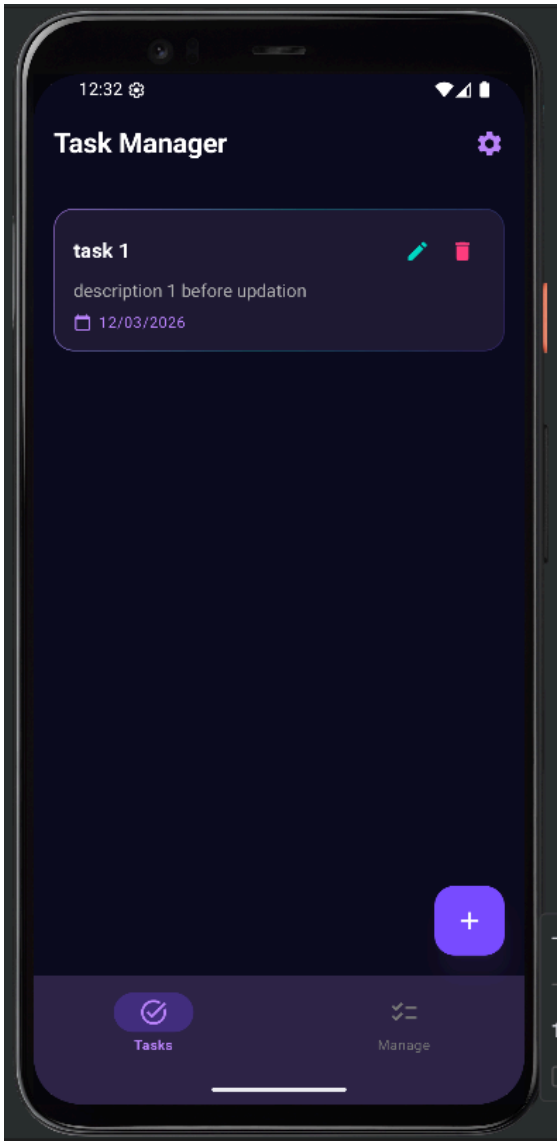


Before updation:(creating task for 1st time)

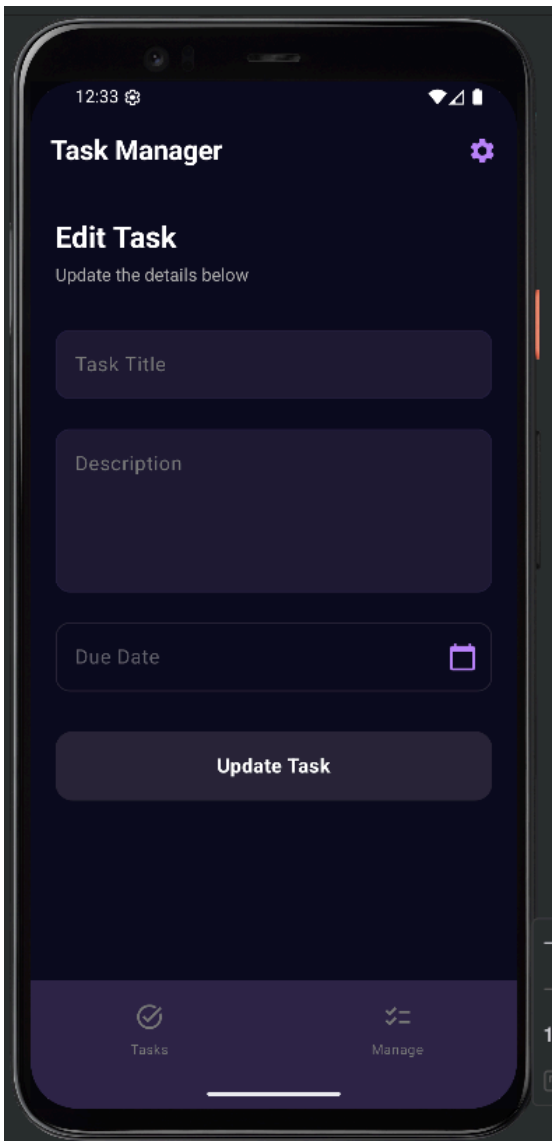


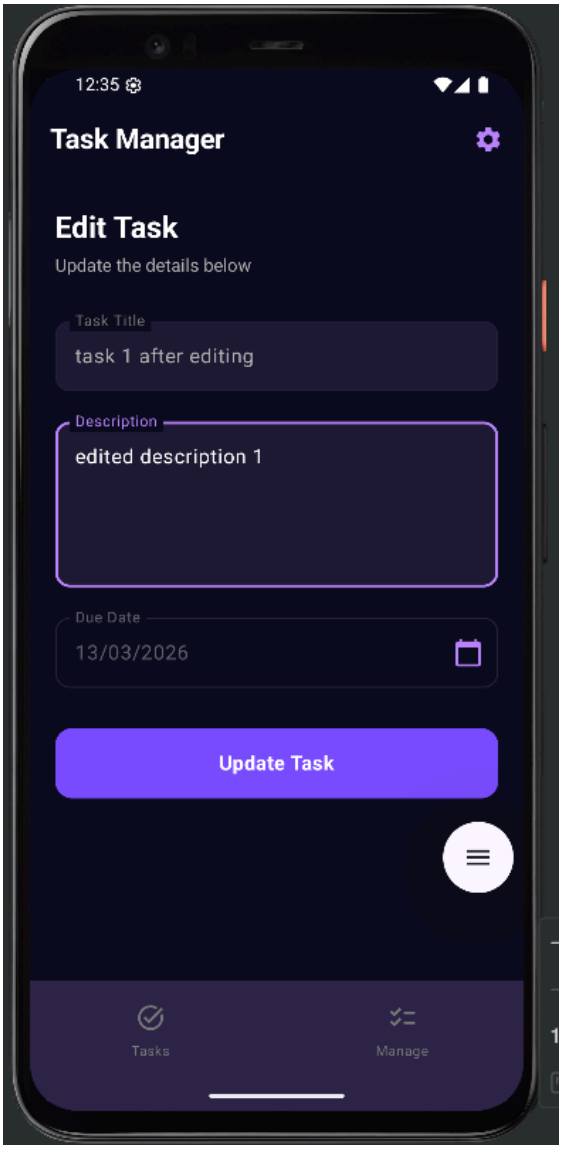


Task Created/view:(before updating)

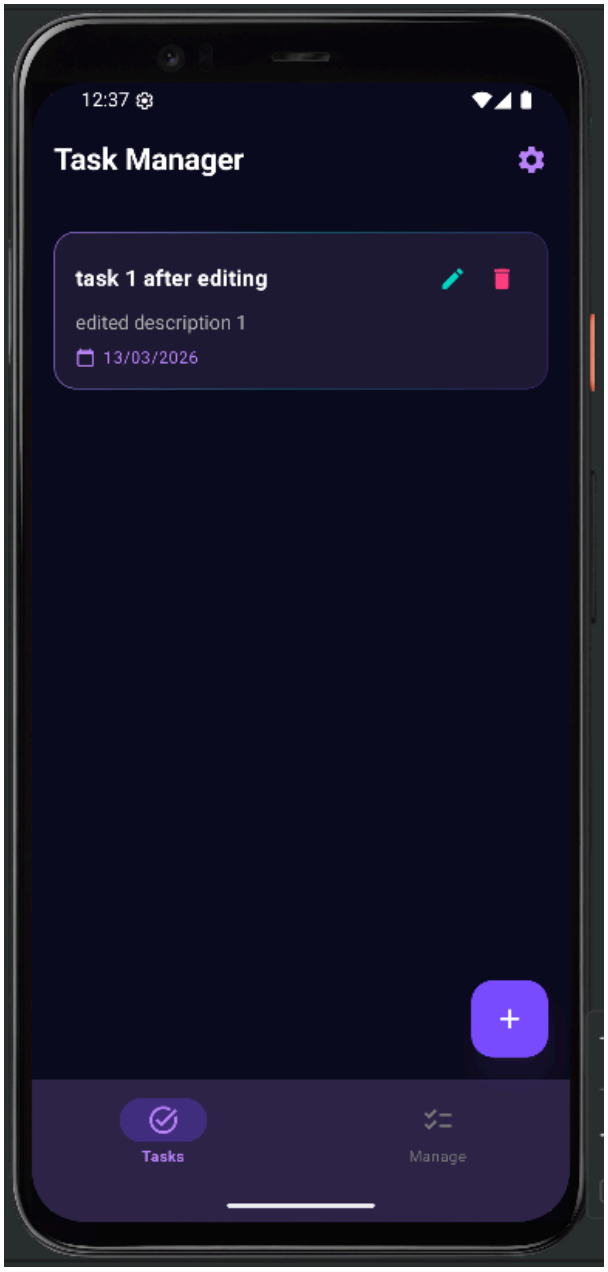


Update/edit task screen:

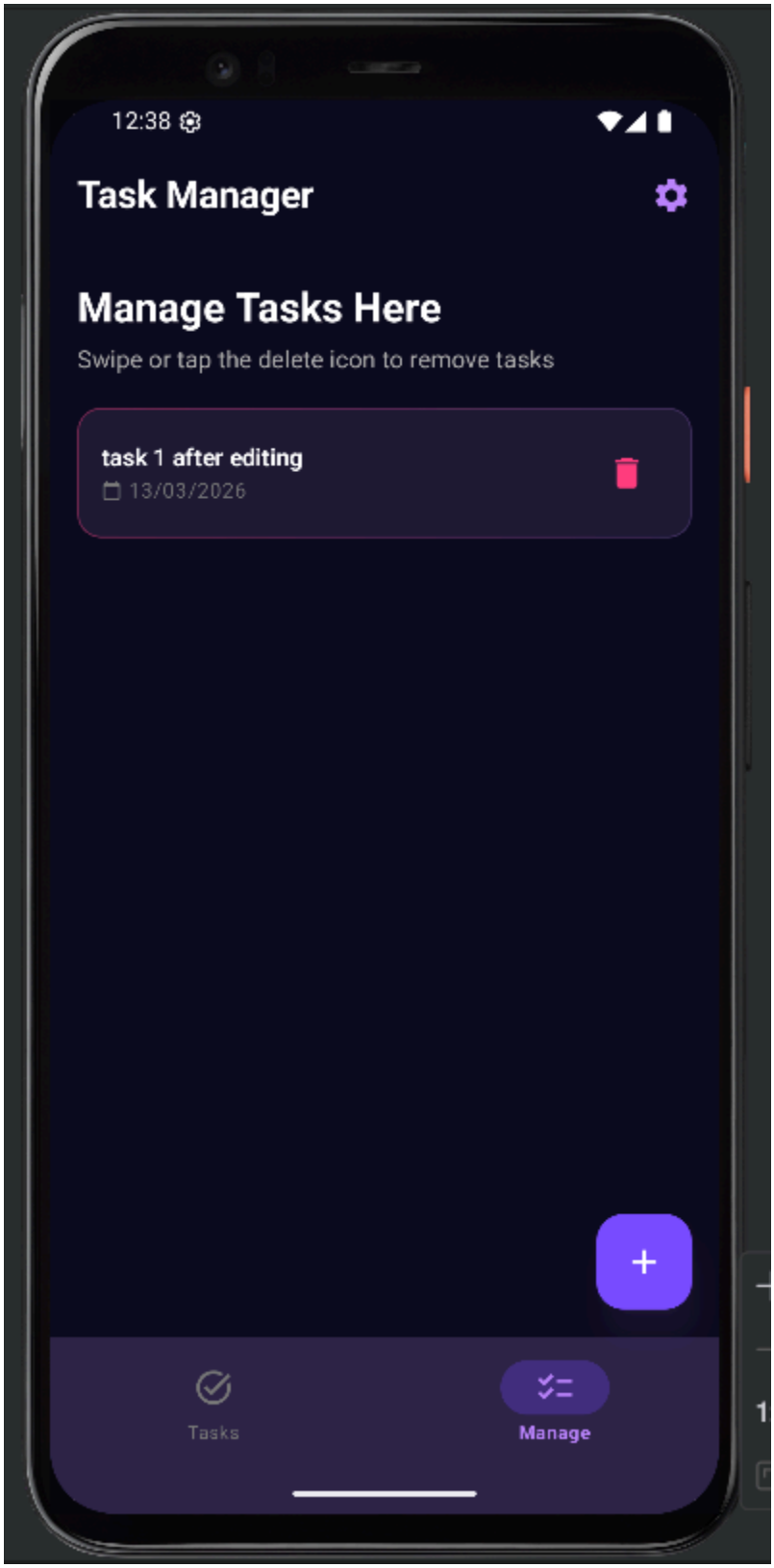
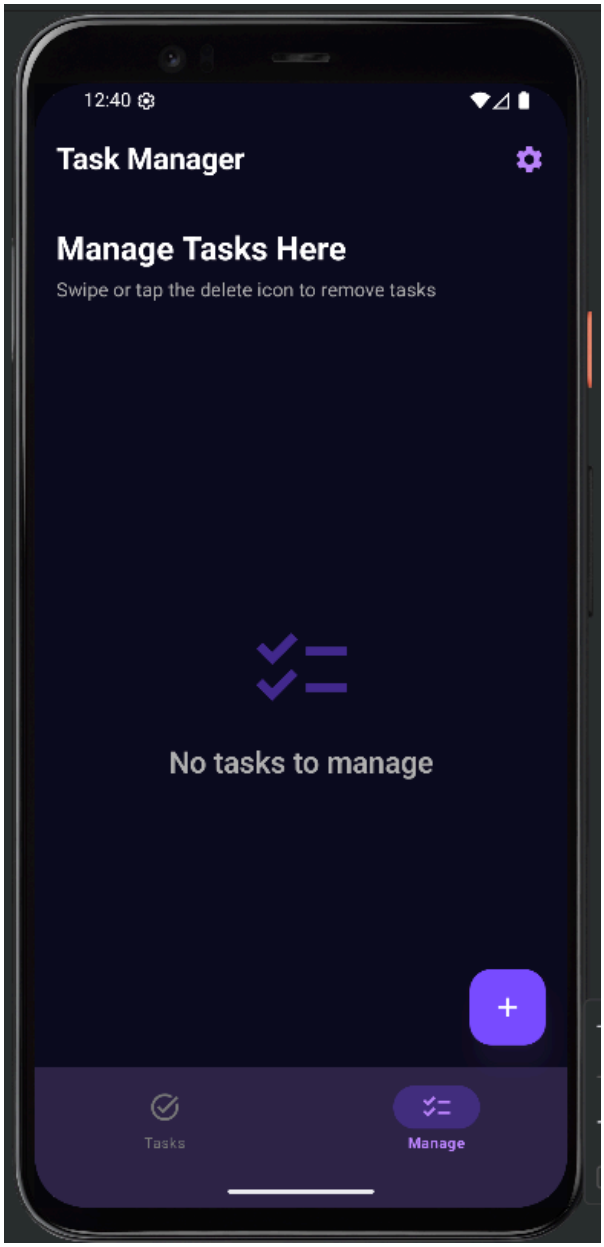




After updation:



]Manage task screen:



Task deleted: